

XRayVision AI

Full-Stack Medical Diagnostic Platform

An AI-powered medical diagnostic web application that analyses chest X-rays, bone fractures, and wound photographs using a multi-model deep learning ensemble (DenseNet121, YOLOv8, ViT) combined with an LLM synthesis layer (OpenRouter GLM 4.5 Air). Built on FastAPI, React 19/TanStack Start, Supabase, Cloudflare Workers and Hugging Face Spaces.

Prepared by

Muhammad Ali Raza & Hamza Afzal

BSSE 8th Semester — Minhaj University Lahore

Supervisor: Maam Misbah

Document Version 2.1.0

Date: June 2026

Table of Contents

- 1. Project Overview** What XRayVision AI is and the problems it solves
- 2. Features Documentation** End-to-end explanation of every feature
- 3. AI System Explanation** How the four models work and the synthesis pipeline
- 4. Technology Stack** Every technology, why it was chosen, and analogies
- 5. Backend Documentation** Architecture, routers, services, security
- 5B. Backend File-by-File Walkthrough** Every important backend file opened and explained
- 6. Frontend Documentation** Pages, components, state, API layer
- 7. Database Documentation** Supabase tables, storage, relationships
- 8. Project Workflow** User journey and backend processing flow
- 9. Running & Deployment** Local setup, environment vars, cloud hosting
- 10. Glossary & References** Technical terms defined in plain English

1. Project Overview

1.1 What Is XRayVision AI?

XRayVision AI is a full-stack medical diagnostic web application that lets doctors, medical students, and healthcare professionals upload medical images — chest X-rays, bone X-rays, and external wound photographs — and receive an AI-powered diagnostic report within seconds. It uses a multi-model deep learning ensemble: each imaging modality is routed to a specialist neural network trained on millions of medical images, and the combined findings are synthesized by a large language model into a readable clinical paragraph.

In simple words: **you upload an image, the system runs several AI models in parallel, and within seconds you get a structured clinical report** — confidence scores, ICD-10 codes, bounding boxes on fractures, urgency rating, recommended actions, and specialist recommendation. All outputs are for educational purposes only, not clinical diagnosis.

1.2 Main Purpose of the System

- Upload medical images (chest X-ray, bone X-ray, wound photo) and receive an AI-powered report
- Detect 18 chest pathologies using DenseNet121 trained on 700,000+ clinical X-rays
- Localise bone fractures with bounding boxes drawn directly on the image using YOLOv8
- Classify external wounds into 6 categories (pressure ulcer, venous ulcer, diabetic foot, etc.) using ViT
- Synthesize all model findings into a readable clinical report via OpenRouter GLM 4.5 Air (free LLM)
- Store all scan history permanently with PDF/JSON export, urgency tracking and ICD-10 codes
- Provide a multilingual health chatbot (English & Urdu) powered by the same LLM
- Generate personalised 7-day diet plans with condition-specific medical safety rules
- Locate nearby hospitals and clinics using the user's GPS coordinates

1.3 Who Can Use This System?

User Type	Typical Use Case	Value Delivered
Medical Students	Learning to interpret X-rays and understand pathologies	AI second opinion with confidence scores and ICD-10 codes
Radiologists	Quick preliminary screening before detailed review	Flags urgent findings; DenseNet121 covers 18 pathologies
Healthcare Professionals	General health queries and patient diet planning	Chatbot + diet planner with clinical safety guardrails
FYP Students	Academic demonstration of full-stack AI integration	Production-grade codebase showcasing real ML deployment

1.4 Real-World Use Cases

Use Case A — A medical student studying chest pathology

Ahmad is a 3rd-year MBBS student. He uploads a chest X-ray from his textbook to XRayVision AI and selects "Chest Pathology". Within 8 seconds, DenseNet121 returns findings: Lung Opacity at 74% confidence (ICD-10: R91.8), Infiltration at 61%. The GLM 4.5 Air synthesis reads: *"The imaging reveals bilateral lung opacity with infiltration patterns consistent with community-acquired pneumonia. Urgency: high. Recommended: Pulmonologist consultation, CBC, CRP."* Ahmad uses this as a study aid alongside his textbook, not as a clinical diagnosis.

Use Case B — A doctor checking a fracture X-ray

Dr. Sana uploads a wrist X-ray for a patient with suspected Colles fracture. She selects "Fracture Detection". YOLOv8 returns a bounding box at 82% confidence labeled "Fracture Detected" in the distal radius region. The synthesis text confirms: *"High-confidence fracture localization in the upper extremity. Urgency: high. Recommended: Orthopedic Surgeon referral, plaster cast, follow-up X-ray."* She downloads the PDF report to attach to the referral.

2. Features Documentation

Every feature is documented with: (a) what problem it solves, (b) the UI on the frontend, (c) the request flow to the backend, and (d) how data is stored and returned.

2.1 AI Image Analysis — The Core Feature

What it does. Takes a user-uploaded medical image and routes it through a specialized AI pipeline: preprocessing → correct model → LLM synthesis → structured report.

Frontend flow.

1. User visits /analyze. A scan-type selector shows three options: Chest Pathology (DenseNet121), Fracture Detection (YOLOv8), External Wound (ViT).
2. User drags-and-drops or selects a PNG/JPG/DICOM image. Optionally adds a session label and clinical notes.
3. On submit, useAnalyze() hook calls analyzeApi.submit(file, scanType, label, notes) from src/lib/api.ts.
4. A full-screen Processing overlay shows while the backend runs (typically 8–15 seconds).
5. On success, TanStack Router navigates to /results/{scanId}. The results page renders: image with bounding box overlays, finding cards with confidence bars, urgency badge, AI synthesis paragraph, ICD-10 codes.

Backend flow.

1. FastAPI receives POST /analyze with the image as multipart/form-data. Rate limit: 10/minute per IP.
2. scan_type is validated (chest | fracture | wound). A UUID scan_id is generated.
3. image_preprocess.py applies CLAHE contrast enhancement and resizes/normalises for the target model.
4. _run_routed_ensemble() uses asyncio.to_thread() to run the correct model in a thread pool. Results are collected with asyncio.gather().
5. synthesize_report() in openrouter_agent.py builds a clinical prompt with all findings and calls OpenRouter GLM 4.5 Air.
6. The image is uploaded to Supabase Storage (xray-images bucket). The full scan record is inserted into the scans PostgreSQL table.
7. A ScanResult JSON object is returned to the frontend.

2.2 Health Chatbot

What it does. Provides a general medical Q&A chatbot that supports both English and Urdu. It recommends doctor types, home remedies, and when to seek emergency care.

Frontend. The /chat page renders a conversation thread with session history. Language toggle switches between English and Urdu. Each session is persisted and restored on next visit.

Backend. POST /chat receives the message, session_id (creates new if null), and language. chatbot_service.py prepends the last 10 messages as conversation context. The system prompt is in English or Urdu based on the language flag. The response is parsed for structured fields: DOCTOR_TYPE: and HOME_REMEDIES: which are

extracted and returned separately for the UI to display in styled cards. Sessions and messages are stored in `chat_sessions` and `chat_messages` tables.

2.3 Diet Plan Generator

What it does. Generates a structured 7-day meal plan personalised to a medical condition, dietary preferences, and restrictions. Adds condition-specific safety rules for hypertension (DASH diet), diabetes (low-GI), and kidney disease (renal dietitian note).

Backend. POST `/diet` calls `diet_service.py` which builds a detailed prompt with medical guardrails and requests a JSON response from OpenRouter. The response is validated — if it does not contain exactly 7 days with complete breakfast/lunch/dinner, a validated hardcoded fallback is returned instead.

2.4 Scan History, PDF Export & JSON Export

What it does. Every analysis result is stored permanently and accessible from the `/history` page. Users can view detailed results, download a formatted PDF report, or export raw findings as JSON for further analysis.

Endpoints. GET `/scans` lists all scans with filtering by type. GET `/scans/{id}` returns the full `ScanResult`. GET `/scans/{id}/report.pdf` generates and streams a formatted PDF. GET `/scans/{id}/export.json` returns the raw findings JSON. DELETE `/scans/{id}` deletes the record and image from storage.

2.5 Clinic Locator

What it does. Finds nearby hospitals and clinics using the browser's GPS coordinates. Returns results with name, address, distance, and a Google Maps link.

Backend. GET `/clinics?lat=X&lon=Y&radius_km=Z` queries the clinics router which searches a database of healthcare facilities and calculates distances using the Haversine formula.

2.6 Dashboard Analytics

What it does. Shows aggregate statistics for the authenticated user: total scans, urgent findings count, average model confidence, average report time, top 5 findings by frequency, and model performance bars.

Backend. GET `/stats` aggregates data from the user's scans in Supabase. It computes: total scans, critical + high urgency count, per-model average confidence from past findings, average `processing_time_ms`, and `finding_distribution` (count per finding name).

3. AI System Explanation

This section explains exactly how the four AI models work — first in everyday language, then technically, with the actual code flow that runs on the server.

3.1 Non-Technical Explanation

Think of XRayVision AI as a team of four specialist doctors looking at your scan. Doctor 1 (DenseNet121) is a chest specialist who has reviewed 700,000 X-rays — he instantly spots pneumonia, effusion, cardiomegaly and 15 other conditions. Doctor 2 (YOLOv8) is an orthopaedic surgeon who circles every fracture he finds directly on the image. Doctor 3 (ViT) is a wound care specialist who classifies skin wounds into 6 types. Once they all report their findings, Doctor 4 (GLM 4.5 Air) — the senior consultant — reads all their notes and writes one clear paragraph explaining what they found, how urgent it is, and what to do next.

3.2 End-to-End Technical Flow

```
Browser → analyzeApi.submit(file, scanType)
        → POST /analyze (JWT required, 10/min rate limit)
        ■
        ▼
        get_current_user_id() ← JWT verified
        ■
        ▼
        image_preprocess.py
        ■■ chest:    CLAHE → grayscale → 224×224 → norm [-1024,1024]
        ■■ fracture: CLAHE in LAB space → original size (YOLO resizes)
        ■■ wound:    convert to RGB PIL Image
        ■
        ▼
        _run_routed_ensemble() ← asyncio.to_thread() per model
        ■■ chest    → chest_model.predict_chest_pathologies()
        ■■ fracture → fracture_model.predict_fractures()
        ■■ wound    → wound_model.predict_wound()
        ■
        ▼
        synthesize_report() ← openrouter_agent.py
        ■■ POST https://openrouter.ai/api/v1/chat/completions
        ■■ model: z-ai/glm-4.5-air:free
        ■
        ▼
        upload_image() → Supabase Storage
        insert_scan()  → Supabase PostgreSQL
        ■
        ▼
        return ScanResult JSON → /results/{scanId}
```

3.3 Model 1 — DenseNet121 (Chest Pathology)

Attribute	Detail
Type	Dense Convolutional Neural Network — 121 layers deep
Training Data	700,000+ clinical chest X-rays (CheXpert + NIH ChestX-ray14 datasets)
Library	TorchXRayVision — open-source medical imaging library (PyTorch)
Task	Multi-label classification: 18 chest pathologies detected simultaneously (independent sigmoids)
Pathologies	Atelectasis, Cardiomegaly, Consolidation, Edema, Effusion, Emphysema, Fibrosis, Hernia, Infiltration, Mass, Nodule, Pleural Thickening, Pneumonia, Pneumothorax, Enlarged Cardiomedastinum, Lung Opacity, Lung Lesion, Fracture
Threshold	Minimum 60% confidence to report a finding (raised from 40% default to reduce false positives)
Max Findings	Top 6 by confidence are kept to avoid burying the most important signals
Input Shape	numpy array (1, 1, 224, 224), float32, normalized to [-1024, 1024] range
Output	18 probability scores (0–100%). Each maps to ICD-10 code + anatomical region

3.4 Model 2 — YOLOv8 (Fracture Detection)

Attribute	Detail
Type	Real-time Object Detection — You Only Look Once v8 (Ultralytics)
Task	Locates fractures in bone X-rays and returns bounding boxes around them
Sensitivity	Threshold set to 15% (very sensitive) to avoid missing subtle fractures
Classes	Fracture Detected, Bone Anomaly, Bone Lesion, Foreign Body, Metallic Implant, Periosteal Reaction, Surgical Hardware
Bounding Box	Output as percentage coordinates (x%, y%, width%, height%) for frontend overlay on original image
Input	BGR numpy array at original resolution — YOLO internally resizes to 960px for inference
Output	List of detections with class, confidence %, bbox coordinates, severity (high/moderate/low), ICD-10

3.5 Model 3 — ViT (Wound Classification)

Attribute	Detail
Type	Vision Transformer (ViT) — transformer architecture applied to image patches
Source	Hugging Face Hub: PayamFard123/dermaintel-wound-classifier

Attribute	Detail
Task	7-class softmax classification of external wound photographs
Classes	Pressure Ulcer (L89.90), Venous Ulcer (I83.009), Arterial Ulcer (I70.25), Diabetic Foot Ulcer (E11.621), Surgical Wound (T81.89XA), Traumatic Wound (T14.8), Normal Skin
How it works	Image is split into 16x16 patches, each treated as a token. Self-attention computes relationships between all patches simultaneously — unlike CNNs which look at local neighborhoods
Output	Always reports top-1 class. Secondary classes shown only if confidence > 15% (above 7-class random baseline)

3.6 Model 4 — OpenRouter GLM 4.5 Air (LLM Synthesis)

What it does. Takes the raw numerical findings from all models and synthesizes them into a readable clinical paragraph with urgency level, recommended actions, and specialist suggestion. The model ID is `z-ai/glm-4.5-air`:free accessed via the OpenRouter API (free tier).

How the prompt is built (`openrouter_agent.py`):

```
SCAN TYPE: CHEST

MODEL FINDINGS:
- Lung Opacity: 74.2% confidence, severity=high, model=DenseNet121,
  region=Lung parenchyma, ICD-10=R91.8
- Infiltration: 61.5% confidence, severity=moderate, model=DenseNet121,
  region=Lung parenchyma, ICD-10=R91.8

Based on these findings, provide a clinical synthesis report.
Return ONLY valid JSON: { urgency, synthesis_text, recommended_actions[], specialist }

RULES:
- urgency must be: critical | high | medium | low | clear
- Cardiologist ONLY if cardiac finding confidence > 70%
- Always include a disclaimer (educational use only)
```

Post-processing: The JSON response is parsed and validated. If a Cardiologist is recommended but no cardiac finding exceeds 70%, the system overrides it with Pulmonologist (the correct default for chest scans). If any finding exceeds 70% confidence but urgency is 'clear', urgency is escalated to 'medium'.

```
# Fallback if OpenRouter fails:
return {
  "urgency": "medium",
  "synthesis_text": "AI synthesis temporarily unavailable. Review findings manually.",
  "recommended_actions": ["Consult a qualified radiologist"],
  "specialist": None
}
```

4. Technology Stack

Every technology was chosen for a specific reason. This section explains each in everyday language first, then with the technical detail.

4.1 Stack at a Glance

Layer	Technology	Role
Frontend Framework	TanStack Start (React 19 SSR)	Pages, routing, server-side rendering
Routing	TanStack Router v1 (file-based)	Each .tsx file = one page route
Data Fetching	TanStack Query v5	Loading states, caching, error handling
Language	TypeScript	Type safety, IDE support, fewer runtime errors
Styling	TailwindCSS v4	Utility classes, no custom CSS needed
UI Components	Shadcn / Radix UI	50+ accessible, pre-built components
Build Tool	Vite + Cloudflare Plugin	Dev server, production bundle
Frontend Hosting	Cloudflare Workers	Global edge network, free tier
Backend Framework	FastAPI 0.115 (Python 3.13)	HTTP server, auto-docs, async support
Server	Uvicorn (ASGI)	Production async server for FastAPI
ML Framework	PyTorch + TorchXRayVision	Powers DenseNet121 and ViT models
Object Detection	Ultralytics YOLOv8	Fracture bounding box detection
Transformers	HuggingFace transformers	ViT wound classifier and optional fracture classifier
Image Processing	OpenCV (cv2) + Pillow	CLAHE, resize, DICOM decode, format conversion
Rate Limiting	slowapi	200/min global, 10/min /analyze
Backend Hosting	Hugging Face Spaces (Docker)	Free CPU tier, ML model support
Database	Supabase PostgreSQL	Relational store with JSON columns
Auth	Supabase Auth + custom JWT	Email/password + HS256 JWT tokens
File Storage	Supabase Storage	X-ray image bucket (xray-images)
LLM API	OpenRouter GLM 4.5 Air	Free-tier synthesis model

4.2 FastAPI (Backend Framework)

What it is. FastAPI is a modern Python web framework for building HTTP APIs. It uses Python type hints to automatically validate request data, generate API documentation, and serialize responses — all with zero extra

code.

Why we chose it. FastAPI is the ideal framework for ML APIs: it runs asynchronously (so heavy model inference doesn't block other requests), integrates natively with Pydantic (for request/response validation), and auto-generates a Swagger UI at /docs that lets you test every endpoint interactively. It is also the fastest Python web framework in benchmarks.

Real-life analogy. FastAPI is like a smart receptionist at a hospital. Each patient (HTTP request) arrives, the receptionist checks their ID (JWT), validates their form (Pydantic), routes them to the right specialist (router), and makes sure the specialist fills out the right paperwork (response schema) before sending them home.

4.3 TanStack Start + React 19 (Frontend)

What it is. TanStack Start is a server-side rendering framework built on React 19. It uses file-based routing (each .tsx file in src/routes/ becomes a URL), server rendering for fast initial page loads, and TanStack Query for intelligent data fetching.

Why we chose it. The combination of TanStack Start + Router + Query gives us type-safe routing, automatic loading/error states, request deduplication, and background data refresh — all features that would require significant boilerplate with older frameworks. React 19's concurrent renderer keeps the UI responsive during heavy operations like processing a 10 MB DICOM file.

4.4 Supabase (Database, Auth & Storage)

What it is. Supabase is a managed PostgreSQL platform that bundles database, authentication, and file storage into one service. It provides a Python SDK (supabase-py) for database operations and a REST API for Auth.

Why we chose it. Instead of managing three separate services (Postgres database, Auth system, S3-like storage), Supabase provides all three with a single set of API keys. Row Level Security (RLS) policies at the database level ensure users can only ever access their own data — a critical security requirement for a medical platform.

5. Backend Documentation

5.1 High-Level Architecture

The backend follows a strict **router** → **service** → **utility** pattern. Every request goes through the same predictable pipeline:



5.2 Folder Structure

```

backend/
  app/
    main.py          ← FastAPI app creation, CORS, routers, model preload
    config.py        ← All env vars via Pydantic Settings
    models/
      schemas.py     ← All Pydantic request/response models
    routers/
      analyze.py     ← POST /analyze – core AI pipeline
      auth.py        ← POST /auth/register, /login, GET /auth/me, PATCH /auth/profile
      scans.py       ← GET /scans, GET /scans/{id}, DELETE, PDF, JSON export
      chat.py        ← POST /chat, GET /chat/sessions
      diet.py        ← POST /diet
      stats.py       ← GET /stats
      clinics.py     ← GET /clinics
    services/
      image_preprocess.py ← CLAHE, resize, DICOM decode, normalization
      chest_model.py    ← DenseNet121 inference + ICD-10 mapping
      fracture_model.py ← YOLOv8 inference + bbox coordinates
      wound_model.py    ← ViT wound classifier
      fracture_classifier.py ← Optional HF fracture presence classifier
  
```

```

openrouter_agent.py ← LLM prompt builder + synthesis engine
openrouter_client.py ← HTTP client for OpenRouter API
chatbot_service.py ← Chatbot system prompts (EN/UR) + response parsing
diet_service.py ← Diet plan generation + safety guardrails
auth_service.py ← JWT creation and verification
utils/
supabase_client.py ← All DB operations + Storage upload

```

5.3 Application Bootstrap (main.py)

When the server starts, FastAPI runs the `lifespan()` context manager. The server immediately starts accepting requests, then launches a background threading.Thread to load the AI models one by one. This prevents the server from appearing unavailable during the 30–60 second model loading window.

```

# backend/app/main.py – startup sequence
@asynccontextmanager
async def lifespan(app: FastAPI):
    if settings.disable_preload:
        # Free-tier hosts: models load on first request
        logger.info('Models will lazy-load on first use')
    else:
        # Full preload in background thread
        thread = threading.Thread(target=_preload_models_background, daemon=True)
        thread.start()
    yield # server is ALREADY accepting requests here

def _preload_models_background():
    load_chest() # DenseNet121 from TorchXRayVision
    load_fracture() # YOLOv8 from fracture_yolov8.pt
    load_wound() # ViT from HuggingFace Hub

```

5.4 Security & Rate Limiting

Security Layer	Implementation
Authentication	Supabase Auth handles email/password signup with bcrypt. Every protected endpoint requires Authorization: Bearer
JWT Verification	auth_service.py decodes HS256 JWT using JWT_SECRET. Server refuses to start if JWT_SECRET is still the insecure default.
Data Isolation	All DB queries include .eq('user_id', user_id) — users cannot access others' scans or chats
Row Level Security	Supabase RLS policies enabled on all tables — second database-level protection layer
Rate Limiting	slowapi: 200 req/min globally, 10/min for POST /analyze, 20/min for POST /chat
CORS	Only FRONTEND_URL is whitelisted in production. Dev mode adds localhost origins.

Security Layer	Implementation
Upload Safety	File bytes are validated non-empty. Content-Type checked. DICOM and standard image formats supported.

5B. Backend File-by-File Walkthrough

This section opens every important backend file and explains what is inside it.

app/config.py — Environment Variables

Uses Pydantic Settings to load all configuration from a .env file. The server refuses to start if JWT_SECRET is still the placeholder value. Accepts both SUPABASE_KEY and SUPABASE_SERVICE_ROLE_KEY so either naming convention works on different hosting platforms.

```
class Settings(BaseSettings):
    supabase_url: str                # e.g. https://xyz.supabase.co
    supabase_key: str                # Service role key (full DB access)
    supabase_anon_key: str           # Anon key (for Auth operations)
    openrouter_api_key: str          # OpenRouter API key
    openrouter_model: str = 'z-ai/glm-4.5-air:free'
    jwt_secret: str                  # HS256 signing key
    yolo_weights_path: str = 'models/fracture_yolov8.pt'
    confidence_threshold: float = 0.40 # base threshold (overridden per model)
    frontend_url: str = 'http://localhost:5173'
    disable_preload: bool = False    # True on free-tier low-RAM hosts
```

app/services/image_preprocess.py — Image Pipeline

Handles all image loading, format conversion, and model-specific preprocessing. Supports PNG, JPG, and DICOM (via pydicom). Each model requires a different input format:

Function	Input	Output	What it does
preprocess_for_chest()	raw bytes	numpy (1,1,224,224) float32	Decode → grayscale → CLAHE → 224×224 → normalize to [-1024,1024]
preprocess_for_yolo()	raw bytes	BGR numpy original size	Decode → CLAHE in LAB color space (preserves color for YOLO)
preprocess_for_vit()	raw bytes	RGB PIL Image	Decode → convert to RGB → PIL (HF processor handles rest)
load_dicom_from_bytes()	DICOM bytes	BGR numpy	pydicom decode → rescale slope/intercept → 8-bit normalization

app/services/chest_model.py — DenseNet121

Lazy-loaded singleton. On first call, downloads and caches the DenseNet121 weights from TorchXRayVision. Runs inference with torch.no_grad() for memory efficiency. Converts sigmoid outputs to percentages and maps each pathology to its ICD-10 code and anatomical region.

```
def predict_chest_pathologies(preprocessed_image, confidence_threshold=0.40):
    model = _get_model() # lazy singleton
```

```
with torch.no_grad():
    tensor = torch.from_numpy(preprocessed_image).float()
    outputs = model(tensor)          # shape: (1, 18)
    probs = outputs.cpu().numpy()[0] # 18 probabilities
    findings = []
    for name, prob in zip(model.pathologies, probs):
        confidence = float(prob) * 100
        if confidence < confidence_threshold * 100:
            continue                # below threshold: skip
        findings.append({
            'name': name,
            'confidence': round(confidence, 1),
            'severity': 'high' if confidence >= 80 else 'moderate' if >= 60 else 'low',
            'model': 'DenseNet121',
            'region': _get_region(name),
            'icd_code': _get_icd_code(name),
        })
    return sorted(findings, key=lambda x: x['confidence'], reverse=True)
```

app/services/fracture_model.py — YOLOv8

Loads the fracture-trained YOLO weights from the path specified in YOLO_WEIGHTS_PATH. Refuses to load generic YOLOv8 weights (yolov8n.pt etc.) unless ALLOW_GENERIC_YOLO_WEIGHTS=true. Bounding boxes are converted from absolute pixel coordinates to percentage-based coordinates so the frontend can overlay them on any image size.

```
def predict_fractures(image, confidence_threshold=0.15):
    model = _get_model() # loads fracture_yolov8.pt
    results = model(image, conf=confidence_threshold, imgsz=960)
    findings = []
    for result in results:
        for box in result.bboxes:
            x1,y1,x2,y2 = box.xyxy[0].cpu().numpy()
            cls_name = result.names[int(box.cls[0])]
            if cls_name.lower() in ('not_fracture', 'normal', 'negative'):
                continue # skip negative class
            findings.append({
                'name': _clean_class_name(cls_name),
                'confidence': round(float(box.conf[0])*100, 1),
                'bbox': { # % of image dimensions for frontend overlay
                    'x': x1/img_w*100, 'y': y1/img_h*100,
                    'w': (x2-x1)/img_w*100, 'h': (y2-y1)/img_h*100,
                },
            })
    return findings or [{'name': 'No fracture box localized', 'confidence': 0}]
```

app/utils/supabase_client.py — Database Operations

Central file for all Supabase interactions. Uses two clients: `get_supabase_client()` (service role key — full access) for most operations, and `get_anon_client()` (anon key) specifically for Auth operations like `sign_up`

and `sign_in_with_password` (which require the anon key by design).

Function	What it does
<code>insert_scan(scan_data)</code>	Insert full scan record (findings JSON, synthesis, image URL, model metadata) into scans table
<code>get_user_scans(user_id)</code>	Fetch all scans for a user, ordered by <code>created_at</code> DESC. Supports <code>scan_type</code> filter and pagination.
<code>get_user_stats(user_id)</code>	Aggregate: total scans, critical count, per-model avg confidence, avg processing time, finding distribution
<code>upload_image(user_id, scan_id, bytes)</code>	Upload X-ray to Supabase Storage at path <code>{user_id}/{scan_id}.png</code> , return public URL
<code>insert_profile(user_id, name, role)</code>	Create profile row after Supabase Auth signup
<code>create_chat_session(user_id, title)</code>	Create new chat session row, return its id
<code>insert_chat_message(session_id, role, content)</code>	Append user or assistant message to a chat session

6. Frontend Documentation

6.1 Folder Structure

src/	
routes/	← One file = one page (TanStack Router)
__root.tsx	← Root layout: auth check, global styles, providers
index.tsx	← Landing page (/)
auth.login.tsx	← Login page (/auth/login)
auth.register.tsx	← Registration page (/auth/register)
auth.forgot-password.tsx	← Password reset page
dashboard.tsx	← Dashboard with stats and recent scans
analyze.tsx	← Image upload and scan type selection
results.\$scanId.tsx	← Results page with bounding boxes + AI report
history.tsx	← Paginated scan history with filters
chat.tsx	← Health chatbot with session management
diet.tsx	← Diet plan generator
clinics.tsx	← GPS-based clinic locator
profile.tsx	← User profile + avatar upload
settings.tsx	← App preferences
lib/	
api.ts	← Central HTTP client – ALL API calls go here
types.ts	← TypeScript types for ScanResult, Finding, etc.
auth-context.tsx	← React context: useAuth() hook for global user state
hooks/	
use-analyze.ts	← TanStack Query mutation for POST /analyze
use-scans.ts	← Query hooks for scan list, individual scan, stats
use-chat.ts	← Query hooks for chatbot sessions and messages
components/	
app/AppShell.tsx	← Sidebar navigation + header wrapper for all pages
ui/	← 50+ Shadcn/Radix components (Button, Card, etc.)
landing/	← Landing page hero, features, X-ray viewer
start.ts	← Cloudflare Workers entry point
router.tsx	← TanStack Router setup

6.2 Page Catalogue

Route	Purpose	Key Components / Hooks
/	Marketing landing page — hero, features, demo	Landing.tsx, Hero3D.tsx, XrayViewer.tsx
/auth/login	Email + password login	authApi.login(), JWT stored in localStorage
/auth/register	New account creation (email, password, role)	authApi.register()
/dashboard	Stats cards, recent scans, model performance bars	useStats(), DashboardStats
/analyze	Scan type picker + image drag-drop + submit	useAnalyze() mutation, navigate to /results/{id}

Route	Purpose	Key Components / Hooks
/results/:id	X-ray with bbox overlays, findings, AI synthesis, urgency	useScans().get(id), PDF download, JSON export
/history	Paginated scan history with type filter	useScans().list(), ScanListResponse
/chat	Chatbot with English/Urdu toggle, session history	useChatSessions(), chatApi.send()
/diet	Medical condition + preferences → 7-day plan	dietApi.generate(), DietPlanResponse
/clinics	GPS → nearby hospitals/clinics with Maps links	clinicApi.search(), GPS Geolocation API
/profile	Update name, role, upload avatar	authApi.updateProfile(), authApi.updateAvatar()

6.3 API Layer — src/lib/api.ts

This single file is the bridge between the React frontend and the FastAPI backend. Every HTTP call in the entire application goes through this file — never directly. It automatically attaches the JWT token from localStorage to every request.

```
// src/lib/api.ts — simplified
const API_URL = import.meta.env.VITE_API_URL.replace(/\/+$/, '');

async function request(path, options = {}): Promise {
  const token = localStorage.getItem('xray_token');
  const headers = token ? { Authorization: `Bearer ${token}` } : {};
  if (!(options.body instanceof FormData))
    headers['Content-Type'] = 'application/json';
  const res = await fetch(`${API_URL}${path}`, { ...options, headers });
  if (!res.ok) throw new Error((await res.json()).detail);
  return res.json();
}

export const analyzeApi = {
  submit: (file, scanType, label?, notes?) => {
    const form = new FormData();
    form.append('file', file);
    form.append('scan_type', scanType);
    return request('/analyze', { method: 'POST', body: form });
  }
};
```

6.4 Authentication Context — src/lib/auth-context.tsx

A React context that stores the current user globally. Any component in the app can call `useAuth()` to get the logged-in user, check if authenticated, or trigger logout. On app load, it reads the stored user from localStorage and validates the token.

7. Database Documentation

The database is PostgreSQL managed by Supabase. All table operations go through `app/utils/supabase_client.py` using the `supabase-py` Python SDK. Row Level Security (RLS) is enabled on all tables.

7.1 Entity Relationship Map

```
profiles (id, full_name, role, avatar_url, settings)
  └─ 1:∞
    └─ scans (id, scan_type, findings[], urgency, image_url,
              agent_synthesis, agent_actions, model_results, created_at)
    └─ chat_sessions (id, user_id, title, created_at)
      └─ 1:∞
        └─ chat_messages (session_id, role, content, created_at)
    └─ [Supabase Auth users] (email, password_hash, created_at)

Storage Buckets:
xray-images/  ← {user_id}/{scan_id}.png
avatars/      ← {user_id}.png
```

7.2 Table Reference

profiles — one row per user, linked to Supabase Auth by id

Column	Type	Description
id	UUID (FK → auth.users)	Matches the Supabase Auth user id
full_name	VARCHAR	Display name
role	VARCHAR	'Medical Student', 'Radiologist', etc.
avatar_url	TEXT	Public URL of uploaded avatar image
settings	JSONB	User UI preferences (theme, language, etc.)
created_at	TIMESTAMPTZ	Account creation time

scans — one row per analysis result

Column	Type	Description
id	UUID	Unique scan id (generated with <code>uuid4()</code>)
user_id	UUID (FK)	Owner — scoped all queries by this
scan_type	VARCHAR	'chest', 'fracture', or 'wound'

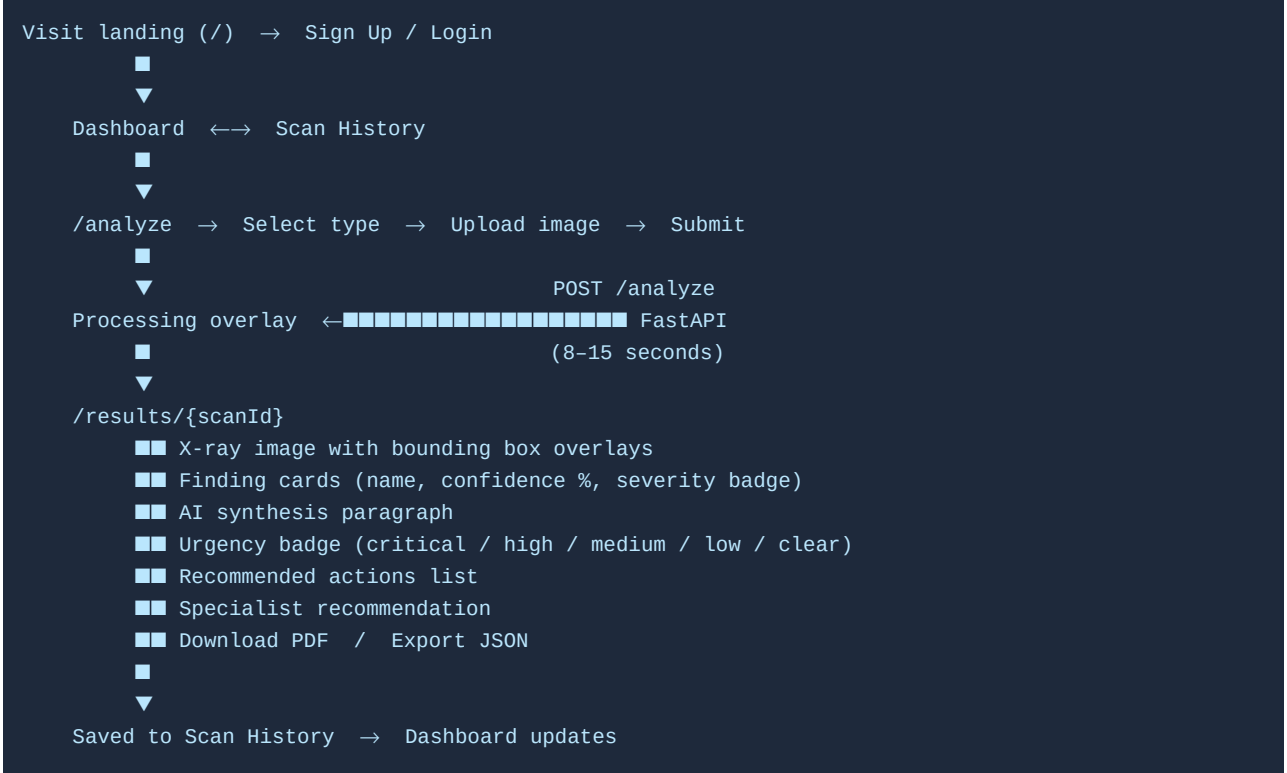
Column	Type	Description
session_label	TEXT	Optional user-provided label for the scan
notes	TEXT	Optional clinical notes from the user
image_url	TEXT	Public URL in Supabase Storage xray-images bucket
urgency	VARCHAR	'critical', 'high', 'medium', 'low', 'clear'
findings	JSONB	Array of Finding objects (name, confidence, severity, bbox, icd_code, model)
agent_synthesis	TEXT	The LLM's clinical paragraph
agent_actions	JSONB	Array of recommended action strings
model_results	JSONB	Metadata: models_run, processing_time_ms, ensemble_mode
created_at	TIMESTAMPTZ	Analysis timestamp

7.3 Supabase Storage Buckets

Bucket	Path Pattern	Access	Content
xray-images	{user_id}/{scan_id}.png	Public URLs (anyone with URL can view)	Uploaded X-ray and wound images
avatars	{user_id}.png	Public URLs with cache-bust timestamp	User profile avatar images

8. Project Workflow

8.1 End-to-End User Journey



8.2 Authentication Flow

Step	What Happens
1. Register	POST /auth/register → Supabase Auth creates account → backend inserts profile row → HS256 JWT (24h) returned
2. JWT stored	Frontend stores token as 'xray_token' in localStorage; user object stored as 'xray_user'
3. API call	Every request attaches Authorization: Bearer header automatically via api.ts
4. Verification	get_current_user_id() FastAPI dependency decodes and verifies JWT on every protected endpoint
5. Logout	localStorage.removeItem('xray_token') + removeItem('xray_user') — no server call needed
6. Password reset	POST /auth/forgot-password → Supabase Auth sends reset email. Always returns success (prevents email enumeration).

8.3 Analysis Pipeline — Detailed Step by Step

Step 1	Image Upload
--------	--------------

User selects scan type (chest/fracture/wound) and uploads an image. DICOM files are supported alongside standard PNG/JPG.

Step 2 Request sent

`analyzeApi.submit()` in `src/lib/api.ts` sends POST `/analyze` as `multipart/form-data`. JWT token auto-attached.

Step 3 Backend validates

FastAPI verifies the JWT, checks `scan_type` is valid, reads file bytes. Rate limit: 10/min.

Step 4 Preprocessing

`image_preprocess.py` decodes the image (OpenCV or `pydicom` for DICOM). CLAHE enhances contrast. Model-specific resize and normalization applied.

Step 5 AI Inference

`_run_routed_ensemble()` routes to ONE model only: DenseNet121 (chest) OR YOLOv8 (fracture) OR ViT (wound). Models run in a thread pool with `asyncio`.

Step 6 LLM Synthesis

`openrouter_agent.py` builds a structured clinical prompt from all findings and calls OpenRouter GLM 4.5 Air. Returns urgency, synthesis text, actions, specialist.

Step 7 Storage

Image uploaded to Supabase Storage. Full scan record (findings JSON, synthesis, image URL) inserted into `scans` table.

Step 8 Response

`ScanResult` JSON returned to frontend. Router navigates to `/results/{scanId}`. Page displays all findings with bounding boxes.

9. Running the Project & Deployment

9.1 Prerequisites

- Python 3.13 and Node.js 18+ installed on your machine
- A Supabase account (free tier) with a project created
- An OpenRouter account (free) to get OPENROUTER_API_KEY
- A Hugging Face account with HF_TOKEN for downloading model weights
- Fracture-trained YOLO weights file (fracture_yolov8.pt) placed in backend/models/

9.2 Backend — First-Time Setup

```
cd backend
python -m venv venv
venv\Scripts\activate          # Windows
# source venv/bin/activate      # macOS/Linux
pip install -r requirements.txt

# Create backend/.env with required variables:
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_KEY=your-service-role-key
SUPABASE_ANON_KEY=your-anon-key
OPENROUTER_API_KEY=sk-or-...
JWT_SECRET=your-random-64-char-secret
HF_TOKEN=hf_...
YOLO_WEIGHTS_PATH=models/fracture_yolov8.pt
FRONTEND_URL=http://localhost:5173

# Start the development server
uvicorn app.main:app --reload --port 8000

# API docs available at: http://localhost:8000/docs
```

9.3 Frontend — First-Time Setup

```
# In project root (where package.json is)
npm install

# Create .env.local
VITE_API_URL=http://localhost:8000

# Start development server
npm run dev

# App runs at: http://localhost:5173
```

9.4 Deployment Architecture

Component	Platform	How it's deployed
Frontend (React)	Cloudflare Workers	npm run build → wrangler deploy. VITE_API_URL set in Cloudflare env vars.
Backend (FastAPI)	Hugging Face Spaces	Dockerfile builds Python env. Pushed to HF Spaces repo. Port 7860 exposed.
Database	Supabase (PostgreSQL)	Managed cloud PostgreSQL. Connection via SUPABASE_URL + service role key.
Image Storage	Supabase Storage	xray-images bucket auto-created if missing. Images stored at user_id/scan_id.png
AI Models (DenseNet, ViT)	Downloaded at startup	HF_TOKEN used to pull from Hugging Face Hub on first server start.
YOLO Weights	Docker image	fracture_yolov8.pt bundled into the Docker image build.

9.5 Environment Variables — Complete Reference

Variable	Where Used	Required	Purpose
SUPABASE_URL	Backend	Yes	Supabase project URL
SUPABASE_KEY	Backend	Yes	Service role key — full DB + storage access
SUPABASE_ANON_KEY	Backend	Yes	Anon key — used for Auth sign_up/sign_in
OPENROUTER_API_KEY	Backend	Yes	API key for GLM 4.5 Air (free tier)
JWT_SECRET	Backend	Yes	Minimum 32-char random string for HS256 signing
HF_TOKEN	Backend	Yes	HuggingFace token for downloading ViT weights
YOLO_WEIGHTS_PATH	Backend	Yes	Path to fracture-trained .pt weights file
FRONTEND_URL	Backend	Yes	Frontend URL for CORS whitelist
DISABLE_PRELOAD	Backend	No	Set 'true' on ≤512MB RAM hosts (HF free tier)
VITE_API_URL	Frontend	Yes	Backend base URL (set in Cloudflare env)

9.6 Troubleshooting

Error	Likely Cause	Fix
JWT_SECRET must be changed	Using default insecure placeholder	Set JWT_SECRET to a real 64-char random hex string in .env
YOLOv8 load error	Wrong weights path or generic weights	Set YOLO_WEIGHTS_PATH to fracture-trained weights or ALLOW_GENERIC_YOLO_WEIGHTS=true for demo

Error	Likely Cause	Fix
OpenRouter 401	Missing or invalid API key	Check OPENROUTER_API_KEY in .env — get free key at openrouter.ai
Supabase 403	Wrong key type	Use service role key for SUPABASE_KEY, not anon key
CORS errors in browser	Frontend URL not whitelisted	Set FRONTEND_URL to your Cloudflare workers domain
Models take 60+ seconds	Cold start on free HF tier	Set DISABLE_PRELOAD=false and wait for background loading to complete

10. Glossary & References

10.1 Technical Terms

Term	Definition
ASGI	Asynchronous Server Gateway Interface — how modern Python web frameworks like FastAPI handle concurrent requests
CLAHE	Contrast Limited Adaptive Histogram Equalization — an image processing technique that enhances local contrast in X-rays, making pathologies more visible to AI models
DenseNet121	A CNN architecture with 121 layers where each layer connects to every other layer. Excellent at medical imaging due to efficient feature reuse.
DICOM	Digital Imaging and Communications in Medicine — the standard file format for medical images (X-rays, CT scans, MRI). XRayVision AI supports DICOM alongside JPG/PNG.
FastAPI	Python web framework. Uses Python type hints to auto-validate requests and auto-generate API documentation.
ICD-10	International Classification of Diseases, 10th revision — standardised medical codes for conditions (e.g., J18.9 = Pneumonia, S02-S92 = Fractures)
JWT	JSON Web Token — a signed authentication token. The server signs it with JWT_SECRET; every subsequent request presents it in the Authorization header.
LLM	Large Language Model — an AI system trained on vast amounts of text. Used here (GLM 4.5 Air) to synthesize model findings into human-readable clinical text.
OpenRouter	An API gateway that provides access to many LLM models including free-tier models like GLM 4.5 Air.
Pydantic	Python data validation library. FastAPI uses it to automatically validate request bodies and shape response data.
RLS	Row Level Security — a database feature in Supabase/PostgreSQL that enforces data access rules at the DB level.
Softmax	A mathematical function that converts a set of numbers into probabilities that sum to 1. Used by the ViT wound classifier to produce class probabilities.
Supabase	Managed PostgreSQL + Auth + Storage platform. XRayVision AI uses it for all three: database, user authentication, and X-ray image storage.
TanStack Start	A full-stack React framework with server-side rendering (SSR), file-based routing, and built-in data fetching via TanStack Query.
ViT	Vision Transformer — a neural network architecture that treats an image as a sequence of patches (like words in a sentence) and uses transformer attention to classify them.
YOLOv8	You Only Look Once v8 — an object detection model that can identify and localize multiple objects in a single forward pass, very fast and accurate.

10.2 External References

- FastAPI documentation — fastapi.tiangolo.com
- TanStack Start / Router / Query — tanstack.com
- TorchXRayVision (DenseNet121) — github.com/mlmed/torchxrayvision
- Ultralytics YOLOv8 — docs.ultralytics.com
- HuggingFace Transformers (ViT) — huggingface.co/docs/transformers
- OpenRouter API — openrouter.ai/docs
- Supabase — supabase.com/docs
- Cloudflare Workers — developers.cloudflare.com/workers
- Hugging Face Spaces — huggingface.co/docs/hub/spaces

10.3 Document Information

Field	Value
Document	XRayVision AI — Enterprise Technical Documentation
Version	2.1.0
Date	June 2026
Project Title	Web-Based AI Chatbot for Automated X-Ray Fracture Detection
Institution	Minhaj University Lahore — School of Software Engineering
Students	Muhammad Ali Raza (2022F-MUL-BSSWE-017) Hamza Afzal (2022F-MUL-BSSWE-027)
Supervisor	Maam Misbah — Lecturer, School of Software Engineering
Backend	FastAPI 0.115 + Python 3.13 — hosted on Hugging Face Spaces
Frontend	React 19 + TanStack Start — hosted on Cloudflare Workers
Database	Supabase PostgreSQL + Storage
AI Models	DenseNet121, YOLOv8, ViT, OpenRouter GLM 4.5 Air